Unit: 2
Java Language Basics

❖ Input and Output Operations

✓ **Reading from keyboard**

- Reading from keyboard means taking input from the user while the program is running.
- Java reads keyboard input using the "Scanner" class with "system.in"

Example:

```
import java.lang.*;
import java.util.Scanner.*;
Class Read{
        public static void main(String[] args){
        Scanner sc= new Scanner(System.in);
        System.out.println("Enter your name: ");
        String name= sc.nextLine();
        System.out.println("Name: "+ name);
        }
}
```

✓ **Java Input Methods (Scanner Class)**

- **nextInt()** – Used to read whole numbers (integers).
- **nextFloat()** – Used to read decimal numbers.
- **nextDouble()** – Used to read more accurate decimal values.
- **next()** – Reads only one word and stops when it finds a space.
- **nextLine()** – Reads the entire line, including spaces.

✓ **Output Operations in Java**

Output operations are used to display data on the screen. In Java, output is mainly performed using System.out.

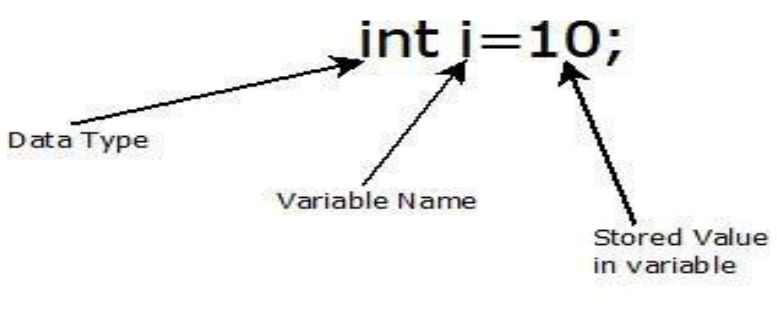
❖ **Methods of Output:**

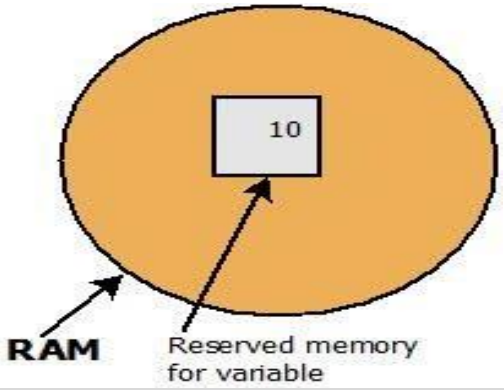
- **System.out.print()** – Prints the output and keeps the cursor on the same line.
- **System.out.println()** – Prints the output and moves the cursor to the next line.

❖ Variables:

A variable is the name of a memory location where data is stored.

Syntax: datatype variableName = Value;





❖ **Rules for Naming Variables in java.**

- Case sensitive [age Vs Age]
- Starting character [a, A, _ , $]
- Subsequent character [0 to 9, _, $]
- No Reserved keywords [Class, static etc]
- Length [No limit but meaningful names needs]
- Naming Convensitions: CamelCase

❖ **Scope of Variables:** Scope of a variable refers to the region of a program where the variable can be accessed or used.

✓ **Types of Variables Based on Scope**

1. Local Variable

- Declared inside a method, constructor, or block.
- Accessible only within that method or block.
- Must be initialized before use.

Example:

```
class Demo {
    public static void main(String[] args) {
        int a = 10;   // Local variable
        System.out.println(a);
    }
}
```

2. Instance Variable

- Declared inside a class but outside all methods.
- Belongs to an object (instance) of the class.
- Each object has its own copy.

Example:

```
class Student {
    int age = 20; // Instance variable
    public static void main(String[] args) {
        Student s = new Student();
        System.out.println(s.age);
    }
}
```

3. Static Variable (Class Variable)

- Declared using the static keyword.
- Shared by all objects of the class.
- Memory is allocated only once.

Example:

```
class Student {
    static String college = "ABC College";
    public static void main(String[] args) {
        System.out.println(college);
    }
}
```

❖ **Primitive Data Types**

Primitive data types are the basic built-in data types in Java used to store simple values such as numbers, characters, and boolean values.

✓ **Java has 8 primitive data types.**

Data Type	Size	Default Value	Example
byte	1 byte	0	byte age = 20;
short	2 bytes	0	short marks = 320;
int	4 bytes	0	int salary = 50000;
long	8 bytes	0L	long population = 8000000000L;
float	4 bytes	0.0f	float price = 99.5f;
double	8 bytes	0.0d	double pi = 3.14159;
char	2 bytes	'\u0000'	char grade = 'A';
boolean	1 bit*	false	boolean isPassed = true;

Note: The exact memory size of boolean is JVM-dependent, though it conceptually stores true or false.

❖ **Categories of Primitive Data Types**

1. Integer Types

- byte
- short
- int
- long

➢ Used to store whole numbers.

2. Floating-Point Types

- float
- double

➢ Used to store decimal numbers.

3. Character Type

- char

➢ Used to store a single character.

4. Boolean Type

- boolean

➢ Used to store true or false.

Example:

```
class PrimitiveDataTypes {
    public static void main(String[] args) {
        byte age = 20;
        int marks = 95;
        long population = 8000000000L;
        float price = 99.5f;
        double pi = 3.14159;
```

```
        char grade = 'A';
        boolean isPassed = true;

        System.out.println(age);
        System.out.println(marks);
        System.out.println(population);
        System.out.println(price);
        System.out.println(pi);
        System.out.println(grade);
        System.out.println(isPassed);
    }
}
```

❖ **Reference Data Types:**

Reference data types are data types that store the memory address (reference) of an object rather than the actual value. These objects are created in the heap memory using the new keyword.

Unlike primitive data types, reference data types can store multiple values and complex data.

✓ **Examples of Reference Data Types**

- Class
- Object
- Array
- String
- Interface
- Enum

Example 1: String

```
public class Demo {
    public static void main(String[] args) {
        String name = "Prince";
        System.out.println(name);
    }
}
```

Output: Prince

Example 2: Array

```
public class Demo {
    public static void main(String[] args) {
        int[] marks = {85, 90, 95};
        System.out.println(marks[0]);
    }
}
```

Output: 85

Example 3: Object

```
class Student {
        String name = "Rahul";
}

public class Demo {
        public static void main(String[] args) {
                Student s = new Student();
                System.out.println(s.name);
        }
}
```

Output: Rahul

✓ **Characteristics of Reference Data Types**

- Store the reference (memory address) of an object.
- Objects are created using the new keyword (except string literals).
- Default value is null.
- Can contain methods and data members.
- Used to represent complex data.

❖ **Type Casting in Java:**

Type Casting is the process of converting one data type into another data type.

Java supports two types of type casting:

1. Implicit Type Casting (Widening)
2. Explicit Type Casting (Narrowing)

1. Implicit Type Casting (Widening)

In implicit type casting, Java automatically converts a smaller data type into a larger data type. No explicit cast is required.

Conversion Order:

byte → short → int → long → float → double

Example:

```
public class Demo {
    public static void main(String[] args) {
        int num = 100;
        double value = num;   // Automatic conversion

        System.out.println(value);
    }
}
```

Output: 100.0

2. Explicit Type Casting (Narrowing)

In explicit type casting, the programmer manually converts a larger data type into a smaller data type using the cast operator.

Syntax:

```
smallerDataType variable = (smallerDataType) largerDataType;
```

Example

```
public class Demo {
    public static void main(String[] args) {
        double value = 99.99;
        int num = (int) value;   // Manual conversion

        System.out.println(num);
    }
}
```

Output: 99

Note: The decimal part is lost during narrowing.

❖ Wrapper Class in Java

A Wrapper Class is a class that converts a primitive data type into an object and vice versa.

Java provides wrapper classes for all 8 primitive data types in the java.lang package.

✓ Primitive Data Types and Their Wrapper Classes

Primitive Data Type	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

✓ **Why Do We Need Wrapper Classes?**

- To convert primitive data types into objects.
- Required when working with Collections (e.g., ArrayList), which store objects.
- Provides useful methods for data conversion and parsing.
- Supports Autoboxing and Unboxing.

1. Autoboxing

Autoboxing is the automatic conversion of a primitive data type into its corresponding wrapper object.

Example

```
public class Demo {
    public static void main(String[] args) {
        int num = 10;
        Integer obj = num;   // Autoboxing

        System.out.println(obj);
    }
}
```

Output: 10

2. Unboxing

Unboxing is the automatic conversion of a wrapper object into its corresponding primitive data type.

Example:

```
public class Demo {
    public static void main(String[] args) {
        Integer obj = 20;
        int num = obj;   // Unboxing

        System.out.println(num);
    }
}
```

Output: 20

✓ **Advantages of Wrapper Classes**

- Allows primitive values to be used as objects.
- Required for Java Collections.
- Provides built-in utility methods like parseInt(), valueOf(), etc.
- Supports Autoboxing and Unboxing.
- Makes data conversion easier.

❖ Operators

Q. What is an Operator?

Ans. An operator is a special symbol that performs some operation on given values or variables.

❖ **Types of Operators:**

1. Arithmetic Operators
2. Relational Operators
3. Logical Operators

4. Assignment Operators
5. Unary Operators
6. Bitwise Operators

1. Arithmetic Operators:

Arithmetic operators are used to perform mathematical operations on variables and values.

Examples: +, -, *, /, %

2. Relational Operators:

Relational operators are used to compare two values or variables.

Examples: ==, !=, >, <, >=, <=

3. Logical Operators:

Logical operators are used to combine multiple conditions and return true or false.

Examples: AND (&&), OR (||), NOT (!)

4. Assignment Operators:

Assignment operators are used to assign values to variables.

Examples: =, +=, -=, *=, /=, %=

5. Unary (Increment/Decrement) Operators:

Unary operators are operators that work on a single operand.

Examples: ++, --

6. Bitwise Operators:

Bitwise operators are used to perform operations on individual bits (binary values).

Examples: AND (&), OR (|), XOR (^), NOT (~), Left Shift (<<), Right Shift (>>)

❖ **Operator Precedence in Java**

- **Operator Precedence** determines the **order in which operators are evaluated** in an expression.
- Operators with **higher precedence** are evaluated before operators with **lower precedence**.

✓ **Operator Precedence Table (Highest to Lowest)**

Precedence	Operators	Description
1 (Highest)	()	Parentheses
2	++, --, +, -, !	Unary Operators
3	*, /, %	Multiplication, Division, Modulus
4	+, -	Addition, Subtraction
5	<, <=, >, >=	Relational Operators
6	==, !=	Equality Operators
7	&&	Logical AND
8	=, +=, -=, *=, /=	Assignment Operators

❖ Conditional Statement

✓ **Conditional Statements:**

A conditional statement is a control structure that executes a block of code only if a specified condition is true.

✓ **Types of Conditional Statements:**

- If Statement
- If-Else Statement
- If-Else-If Ladder
- Nested If-Else
- Ternary Operator
- Switch Statement

1. **If Statement:**

An if statement is used to check a condition. If the condition is true, the block of code executes.

Syntax:

```
if (condition) {
  // code to execute
}
```

2. If-Else Statement:

An if-else statement is used to check a condition and execute one block if the condition is true; otherwise, it executes another block.

Syntax:

```
if (condition) {
    // code to execute if the condition is true
} else {
    // code to execute if the condition is false
}
```

3. Else-If Ladder:

An else-if ladder is used to check multiple conditions one by one. If one condition is true, its corresponding block of code executes.

Syntax:

```
if (condition1) {
    // code to execute if condition1 is true
} else if (condition2) {
    // code to execute if condition2 is true
} else if (condition3) {
    // code to execute if condition3 is true
} else {
    // code to execute if none of the conditions are true
}
```

4. Nested If-Else:

Nested if-else means using an if-else statement inside another if or else block.

Syntax:

```
if (condition1) {
    if (condition2) {
        // code to execute if both conditions are true
    } else {
        // code to execute if condition2 is false
    }
} else {
    // code to execute if condition1 is false
}
```

5. Ternary Operator:

A ternary operator is a one-line conditional expression used to assign a value based on a condition.

Syntax:

```
variable = (condition) ? value_if_true : value_if_false;
```

Example:

```
public class Main {
    public static void main(String[] args) {
        int days = 35;
        String status = (days >= 30) ? "Regular" : "Irregular";
        System.out.println(status);
    }
}
```

6. Switch Statement:

A switch statement is used to select one block of code from multiple options based on the value of an expression.

Syntax:

```
switch (expression) {
        case value1:
            // code to execute
            break;
        case value2:
            // code to execute
            break;
        default:
            // code to execute if no case matches
}
```

❖ **Loops in Java**

- A loop is a control structure that repeats a set of statement as long as a given condition is true.
- Loops in Java are used to repeat a block of code multiple times until a condition is met. They're fundamental for tasks like iterating over arrays, processing data, or repeating actions.

❖ **Types of loops in Java**

1. for loop
2. while loop
3. do-while loop

1. **for loop: Used when you know how many times you want to run the loop.**

Syntax:

```
for( initialization, condition, update){
        // code
         //code
}
```

2. While Loop:

A while loop is used when you don't know the exact number of iterations, and the loop depends on a condition.

Syntax:

```
while(condition) {
      // code to execute
}
```

Example:

```
 int i = 1;
while(i <= 5) {
      System.out.println(i);
      i++;
}
```

3. Do-While Loop:

A do-while loop executes the code at least once, even if the condition is false, because the condition is checked after execution.

Syntax:

```
do {
     // code to execute
} while(condition);
```

Example:

```
int i = 1;
do {
   System.out.println(i);
   i++;
} while(i <= 5);
```

1. break keyword

The break keyword is used to stop the loop immediately.

Example:

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) {
      break;
    }
   System.out.println(i); // 1 2
}
```

2. continue keyword

The continue keyword is used to skip the current iteration and move to the next iteration.

Example:

```
for (int i = 1; i <= 5; i++) {
      if (i == 3) {
      continue;
    }
   System.out.println(i); // 1 2 4 5
}
```

Important Questions

1. Explain the types of variables in Java with examples.
2. Differentiate between Primitive and Reference Data Types with examples.
3. Explain Type Casting in Java with examples.
4. What are Wrapper Classes? Explain Autoboxing and Unboxing with suitable examples.
5. Explain the different types of operators in Java with examples.
6. What is Operator Precedence? Explain with suitable examples.
7. Explain the decision-making statements in Java (if, if-else, nested if, and switch) with examples.
8. Explain the looping statements in Java (for, while, do-while, and for-each) with examples.
9. Differentiate between for, while, and do-while loops with suitable examples.
10. Explain the break and continue statements with suitable examples.

*****